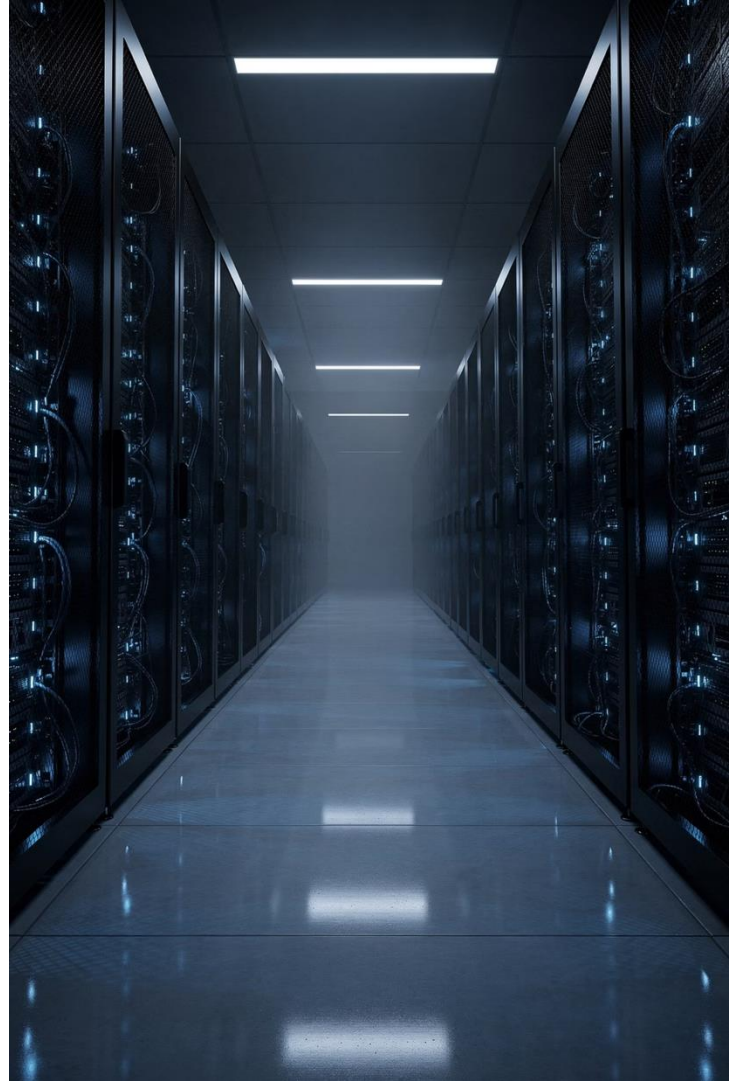


Sesión 5: Monitorización, Auditoría y Simulación de Ataques

Módulo IV · Seguridad Aplicada a Entornos Java & .NET

DURACIÓN: 2 H 40 MIN

SESIÓN 5 DE 5



Objetivos de la sesión

Al finalizar esta sesión serás capaz de diseñar estrategias de auditoría robustas, integrar herramientas de monitorización en pipelines reales y ejecutar simulaciones de ataque controladas para identificar y mitigar vulnerabilidades en arquitecturas backend modernas.



Auditoría

Diseñar un sistema de logging de acceso y registro de eventos de seguridad con trazabilidad completa.



Monitorización

Integrar herramientas especializadas para la detección y alertas en tiempo real sobre anomalías.



Simulación

Ejecutar y analizar ataques controlados sobre APIs y servicios backend usando metodologías estándar.



Laboratorio

Revisar una arquitectura completa, identificar riesgos y proponer medidas de mejora concretas.

Estructura de la sesión

01

Bloque 1 · 60 min

Auditoría y trazabilidad — Logging de acceso, registro de eventos de seguridad e integración con herramientas de monitorización.

02

Bloque 2 · 50 min

Simulación de ataques — Metodologías de evaluación, herramientas de análisis e interpretación de resultados.

03

Bloque 3 · 50 min

Laboratorio final integrador — Revisión de arquitectura real, identificación de riesgos y propuesta de mejoras.

BLOQUE 1

Auditoría y Trazabilidad

La capacidad de reconstruir exactamente qué ocurrió, cuándo y quién lo provocó es la base de cualquier estrategia de respuesta a incidentes. Sin trazabilidad, la seguridad es ciega.



¿Por qué es crítica la auditoría?

Sin auditoría

- No se puede detectar cuándo ocurrió una brecha
- Imposible determinar qué datos fueron accedidos
- Sin evidencias para análisis forense
- Incumplimiento de normativas (GDPR, PCI-DSS, SOC2)

Con auditoría completa

- Reconstrucción precisa de la línea temporal del ataque
- Identificación del vector de entrada y recursos comprometidos
- Evidencias admisibles en procesos legales
- Base para la mejora continua de controles

Regulaciones como el RGPD exigen que las organizaciones demuestren quién accedió a qué datos personales y cuándo. El logging no es opcional: es una obligación legal y técnica.

Logging de acceso: principios fundamentales

Un sistema de logging de acceso efectivo debe capturar suficiente contexto para reconstruir cualquier operación, sin almacenar información sensible que pueda convertir los propios logs en un vector de ataque.

¿Qué registrar?

Identidad del usuario o servicio, timestamp con zona horaria UTC, recurso accedido, método HTTP, IP de origen, resultado (éxito/fallo), duración de la operación y correlation ID.

¿Qué NO registrar?

Contraseñas, tokens completos, números de tarjeta, datos de salud u otros datos personales sensibles. Los logs deben ser auditables sin ser un riesgo por sí mismos.

Formato estructurado

Usar siempre JSON estructurado con campos predefinidos. Facilita la indexación, búsqueda y correlación automática en SIEM. Evitar mensajes de texto libre y no estructurado.

Anatomía de un evento de log de seguridad

Un log de seguridad bien estructurado incluye campos que permiten la correlación y el análisis automatizado. Este ejemplo muestra un intento de autenticación fallido capturado en formato JSON:

```
{
  "timestamp": "2024-11-14T08:32:11.423Z",
  "level": "WARN",
  "event_type": "AUTH_FAILURE",
  "correlation_id": "req-8f2a1b3c",
  "user_id": "usr_0042",
  "client_ip": "203.0.113.45",
  "user_agent": "Mozilla/5.0 ...",
  "resource": "/api/v1/accounts",
  "http_method": "POST",
  "status_code": 401,
  "reason": "INVALID_CREDENTIALS",
  "attempt_count": 3,
  "geo_country": "RU"
}
```

⚠ El campo `attempt_count` y `geo_country` son clave para detectar ataques de fuerza bruta distribuidos. La correlación de estos campos activa alertas automáticas en el SIEM.

Implementación en Spring Boot

Spring Security expone filtros y eventos que permiten interceptar y registrar todas las operaciones de autenticación y autorización. La integración con SLF4J + Logback o Log4j2 es directa y configurable.

```
@Component
public class SecurityAuditListener
    implements ApplicationListener<AbstractAuthenticationEvent> {

    private static final Logger audit =
        LoggerFactory.getLogger("SECURITY_AUDIT");

    @Override
    public void onApplicationEvent(AbstractAuthenticationEvent event) {
        String principal = event.getAuthentication()
            .getName();

        if (event instanceof AuthenticationSuccessEvent) {
            audit.info("{\"event\": \"LOGIN_OK\", \"user\": \"{}}\", principal);
        } else if (event instanceof AbstractAuthenticationFailureEvent e) {
            audit.warn("{\"event\": \"LOGIN_FAIL\", \"user\": \"{}}\", \"reason\": \"{}}\",
                principal, e.getException().getMessage());
        }
    }
}
```

① Utiliza un logger dedicado (`SECURITY_AUDIT`) separado del logger de aplicación. Esto permite enviar los eventos de seguridad a un destino diferente (Elasticsearch, Splunk) sin mezclarlos con logs de negocio.

Implementación en ASP.NET Core

ASP.NET Core permite combinar middleware personalizado con el sistema de eventos de ASP.NET Identity para capturar operaciones de autenticación, acceso a recursos protegidos y cambios de configuración críticos.

```
public class SecurityAuditMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<SecurityAuditMiddleware> _logger;

    public SecurityAuditMiddleware(RequestDelegate next,
        ILogger<SecurityAuditMiddleware> logger)
    {
        _next = next; _logger = logger;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        var sw = Stopwatch.StartNew();
        await _next(context);
        sw.Stop();

        _logger.LogInformation(
            "AUDIT {@Audit}",
            new {
                User = context.User?.Identity?.Name ?? "anonymous",
                Path = context.Request.Path.Value,
                Method = context.Request.Method,
                Status = context.Response.StatusCode,
                Duration = sw.ElapsedMilliseconds,
                IP = context.Connection.RemoteIpAddress?.ToString()
            });
    }
}
```

Niveles de log para eventos de seguridad

Asignar el nivel correcto a cada evento es fundamental para que los sistemas de alertas puedan priorizar correctamente sin generar ruido excesivo.

Nivel	Eventos de seguridad típicos	Acción recomendada
DEBUG	Evaluación de permisos individuales, resolución de roles	Solo en entorno de desarrollo, nunca en producción
INFO	Login correcto, acceso a recursos autorizados, logout	Retención 90 días, análisis de comportamiento
WARN	Credenciales incorrectas, token expirado, acceso denegado	Alerta si supera umbral en ventana temporal
ERROR	Fallo al validar firma JWT, error de certificado SSL	Alerta inmediata, revisión en menos de 1 hora
FATAL	Intento de manipulación de almacén de claves, brecha activa	Alerta crítica, respuesta a incidentes automática

Registro de eventos de seguridad: taxonomía

Más allá del logging de acceso HTTP, es necesario registrar eventos de negocio de seguridad con una taxonomía bien definida. Esto permite búsquedas eficientes y correlación en herramientas SIEM.



Identidad y acceso

Creación/eliminación de cuentas, cambios de contraseña, activación de MFA, bloqueo por intentos, elevación de privilegios.



Autorización

Acceso a recursos sensibles, denegaciones de permisos, cambios en roles o políticas de acceso, acceso fuera de horario habitual.



Datos

Exportaciones masivas, acceso a datos PII, modificaciones en registros críticos, borrados irreversibles.

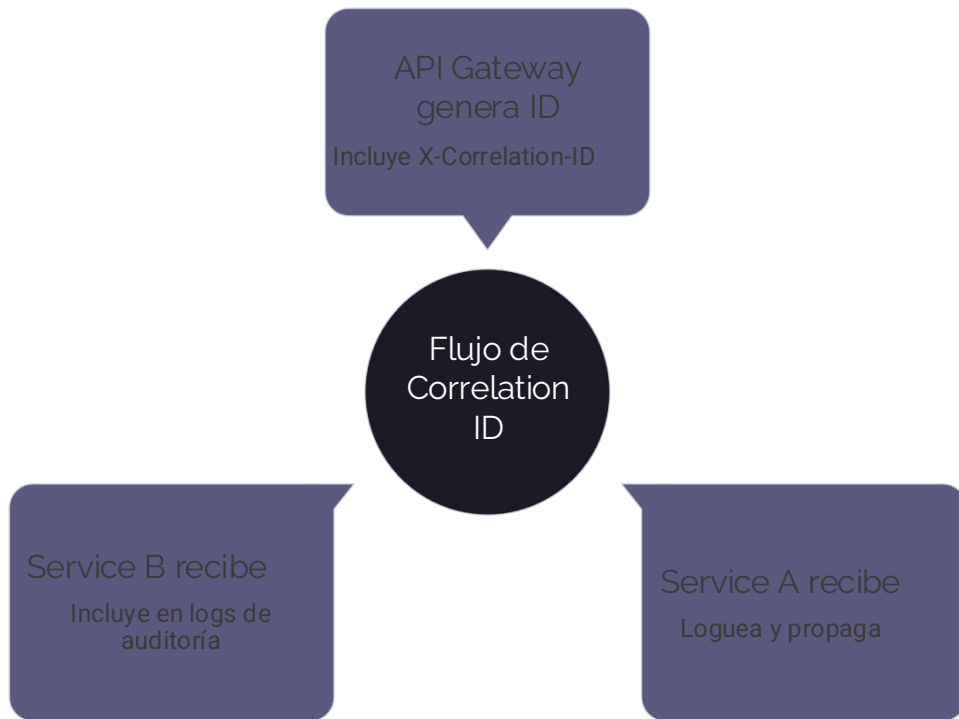


Configuración

Cambios en parámetros de seguridad, rotación de secretos, modificación de certificados, desactivación de controles.

Trazabilidad distribuida: el Correlation ID

En arquitecturas de microservicios, una única operación de usuario puede atravesar 5, 10 o más servicios. Sin un identificador de correlación propagado correctamente, reconstruir el flujo completo de una petición es prácticamente imposible.



El X-Correlation-ID debe generarse en el API Gateway (o en el primer punto de entrada) y propagarse automáticamente en cada llamada saliente. Todas las trazas de todos los servicios quedan enlazadas bajo el mismo identificador, permitiendo reconstruir el flujo completo en el SIEM.

Integración con herramientas de monitorización

Los logs aislados en ficheros locales tienen escaso valor operacional. La integración con plataformas centralizadas de monitorización transforma los eventos en inteligencia accionable.



ELK Stack

Elasticsearch + Logstash + Kibana. Stack open source ampliamente adoptado. Logstash ingesta y normaliza, Elasticsearch indexa y Kibana visualiza. Ideal para volúmenes medianos-altos con control total.



Grafana + Loki

Alternativa ligera al ELK. Loki indexa solo metadatos (no el contenido completo), reduciendo drásticamente el coste de almacenamiento. Grafana unifica métricas y logs en un único panel.



Splunk SIEM

Plataforma enterprise. Correlación de eventos, reglas SPL para detección de amenazas, inteligencia de amenazas integrada y playbooks de respuesta automática. Estándar en entornos regulados.

Configuración de alertas de seguridad

Las alertas deben ser precisas, contextualizadas y accionables. Una alerta mal configurada genera fatiga de alertas, que es uno de los principales motivos por los que los equipos ignoran señales de ataque reales.

1

Define umbrales con contexto

No alertes por número absoluto sino por anomalía. 100 logins fallidos de una IP en 5 minutos es diferente a 100 logins fallidos distribuidos entre 1000 usuarios en 1 hora.

2

Prioriza por impacto

Clasifica alertas en P1 (respuesta inmediata), P2 (revisión en 1h) y P3 (análisis diario). Vincula la prioridad al tipo de recurso afectado y a la criticidad del dato.

3

Enriquece el contexto

Agrega geolocalización, reputación de IP, historial del usuario y clasificación del recurso. Una alerta enriquecida se resuelve en minutos; una alerta cruda puede tardar horas.

4

Automatiza la respuesta

Usa playbooks (SOAR) para acciones automáticas: bloqueo de IP, invalidación de tokens, cuarentena de cuenta, notificación al equipo de guardia.

Retención, protección e integridad de logs

Los logs de seguridad son evidencias. Deben estar protegidos contra modificación, eliminación accidental o maliciosa, y mantenerse durante el período que exija la normativa aplicable.

Requisitos de retención

- **RGPD:** Mínimo recomendado 12 meses activos + 24 meses archivo
- **PCI-DSS:** 12 meses, 3 meses accesibles en línea
- **SOC 2:** Alineado con el período de auditoría (12 meses)
- **Sector bancario (EBA):** Hasta 7 años para ciertos eventos

Protección de la integridad

- Almacenamiento en destinos de solo escritura (WORM)
- Firma digital o hash encadenado de los registros
- Separación de acceso: el servicio puede escribir pero no leer ni borrar
- Copia de seguridad cifrada en cuenta cloud separada
- Alertas ante gaps temporales o reducción súbita de volumen



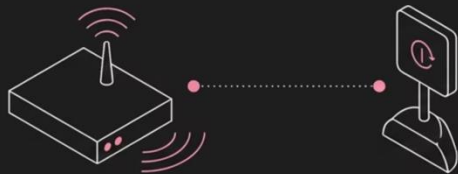
BLOQUE 2

Simulación de Ataques y Análisis de Vulnerabilidades

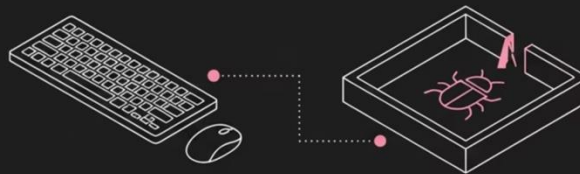
Conocer cómo piensan los atacantes es indispensable para construir defensas efectivas. La simulación controlada convierte el conocimiento teórico en resiliencia real.

Tipos de evaluación de seguridad

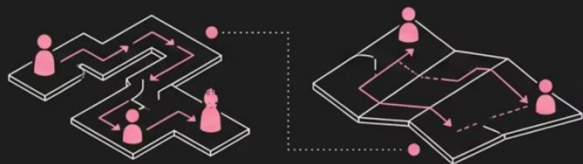
Existen distintos enfoques para evaluar la seguridad de un sistema, cada uno con un alcance, profundidad y perfil de conocimiento previo diferente. Elegir el adecuado depende del objetivo y del contexto.



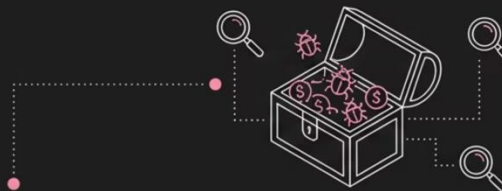
Vulnerability Assessment: Escaneo automatizado de vulnerabilidades conocidas. Bajo coste y rápido.



Penetration Testing: Simulación manual de ataque real. Alcance definido, mayor profundidad.



Red Team Exercise: Simulación de adversario persistente. Sin límite de tiempo, máxima profundidad.



Bug Bounty: Investigadores externos buscan vulnerabilidades de forma continua. Pago por hallazgo.

Metodologías de referencia

Las evaluaciones de seguridad profesionales siguen metodologías estandarizadas que garantizan cobertura, reproducibilidad y comparabilidad de resultados entre evaluaciones sucesivas.

→ OWASP Testing Guide (OTG)

La metodología de referencia para pruebas de seguridad en aplicaciones web y APIs. Cubre más de 90 categorías de prueba organizadas en familias. Versión actual: OTG v4.2. Base para la mayoría de los pentests de aplicación.

→ PTES — Penetration Testing Execution Standard

Define las fases de un pentest completo: Pre-engagement, Intelligence Gathering, Threat Modeling, Vulnerability Analysis, Exploitation, Post-Exploitation, Reporting. Muy usado para definir el alcance contractual.

→ CVSS — Common Vulnerability Scoring System

Sistema de puntuación estándar (0–10) para cuantificar la gravedad de vulnerabilidades. Considera vectores de ataque, complejidad, privilegios requeridos e impacto en confidencialidad, integridad y disponibilidad.

Fases de un pentest de aplicación backend



Cada fase tiene entregables bien definidos. El reporte final debe incluir la puntuación CVSS de cada hallazgo, los pasos de reproducción y la recomendación de remediación con nivel de prioridad. La validación post-remediación es parte del proceso.

Herramientas de análisis: visión general

El ecosistema de herramientas para evaluación de seguridad es amplio. Para aplicaciones backend, las más relevantes se agrupan en tres categorías según la técnica que aplican.

SAST — Análisis estático

Analiza el código fuente sin ejecutarlo. Detecta patrones vulnerables, inyecciones, uso inseguro de criptografía y problemas de configuración. Se integra en el pipeline de CI/CD.

- SonarQube, Checkmarx
- Semgrep, SpotBugs
- CodeQL (GitHub Advanced Security)

DAST — Análisis dinámico

Prueba la aplicación en ejecución enviando peticiones maliciosas y analizando las respuestas. Detecta vulnerabilidades que solo se manifiestan en tiempo de ejecución.

- OWASP ZAP (Zed Attack Proxy)
- Burp Suite Professional
- Nikto, Nuclei

SCA — Dependencias

Analiza las dependencias del proyecto buscando versiones con CVEs conocidos. Esencial dado que la mayoría del código de una aplicación moderna es de terceros.

- OWASP Dependency-Check
- Snyk, Trivy
- GitHub Dependabot

OWASP ZAP: proxy de ataque en la práctica

OWASP ZAP es la herramienta DAST de referencia open source. Actúa como proxy interceptor entre el cliente y el servidor, permitiendo inspeccionar, modificar y repetir peticiones, así como ejecutar escaneos automatizados.

Capacidades principales

- Interceptación y modificación de peticiones HTTP/HTTPS
- Spider activo para descubrimiento de endpoints
- Escaneo activo de vulnerabilidades (SQLi, XSS, XXE...)
- Fuzzing de parámetros con payloads personalizados
- Modo daemon para integración en CI/CD
- Generación de reportes en HTML, JSON, XML

Integración en pipeline

- Imagen Docker oficial disponible
- API REST para automatización completa
- Plugin para Jenkins, GitHub Actions, GitLab CI
- Modo baseline scan para PRs (escaneo rápido)
- Modo full scan para ramas principales

Burp Suite: análisis manual avanzado

Burp Suite Professional es la herramienta estándar de la industria para pentesting manual de aplicaciones web y APIs. Su punto fuerte es la capacidad de análisis profundo y la extensibilidad mediante plugins (BApps).



Repeater

Permite repetir y modificar manualmente peticiones individuales para probar variantes de payloads y analizar diferencias en respuestas. Fundamental para la explotación manual.



Intruder

Automatiza ataques de fuerza bruta, fuzzing de parámetros y enumeración. Permite definir posiciones de inyección y listas de payloads con filtrado de resultados por diferencias en respuesta.



Scanner

Escaneo activo automatizado que detecta vulnerabilidades conocidas. Genera un informe detallado con descripción, nivel de confianza, evidencias y recomendaciones de remediación.



Extensiones (BApps)

Marketplace de extensiones para casos específicos: JWT Editor, Active Scan++, Autorize (pruebas de control de acceso), JS Miner, GraphQL Raider.

Semgrep y SonarQube: SAST en el pipeline

Las herramientas SAST integradas en CI/CD permiten detectar vulnerabilidades antes de que el código llegue a producción, reduciendo drásticamente el coste de remediación.

Semgrep

Motor de análisis semántico basado en patrones. Permite escribir reglas personalizadas en YAML para detectar patrones específicos del stack tecnológico de la organización. Muy rápido y con bajo número de falsos positivos.

```
rules:
- id: sql-injection-jdbc
  pattern: |
    $STMT.executeQuery(
      "..." + $INPUT + "...")
  message: Posible SQL Injection
  severity: ERROR
  languages: [java]
```

SonarQube

Plataforma de calidad de código con módulo de seguridad (Security Hotspots). Cubre reglas OWASP Top 10, CWE Top 25 y SANS Top 25. Quality Gates bloquean el merge si se detectan vulnerabilidades bloqueantes.

- Soporte nativo para Java, C#, JS, Python...
- Integración con GitHub, GitLab, Bitbucket, Azure DevOps
- Taint analysis para rastrear flujos de datos peligrosos
- Dashboard de deuda de seguridad acumulada

Simulación de ataques comunes en APIs

Más allá de los escáneres automáticos, es fundamental conocer cómo ejecutar manualmente los ataques más frecuentes en APIs backend para comprender su impacto real y validar los controles defensivos.

1

SQL Injection en parámetros

Inyectar payloads como `' OR '1'='1 o '; DROP TABLE--` en campos de búsqueda, filtros y ordenación. Usar `sqlmap` para automatizar la detección y extracción.

2

JWT manipulation

Probar algoritmo `none`, cambiar HS256 a RS256 con clave pública como secreto, modificar claims de rol. Herramienta: JWT Editor de Burp o `jwt_tool`.

3

IDOR — Broken Object Level Authorization

Sustituir IDs de recursos propios por ajenos en endpoints REST (`/api/orders/1234` → `/api/orders/1235`). Automatizable con `Autorize` (BApp de Burp Suite).

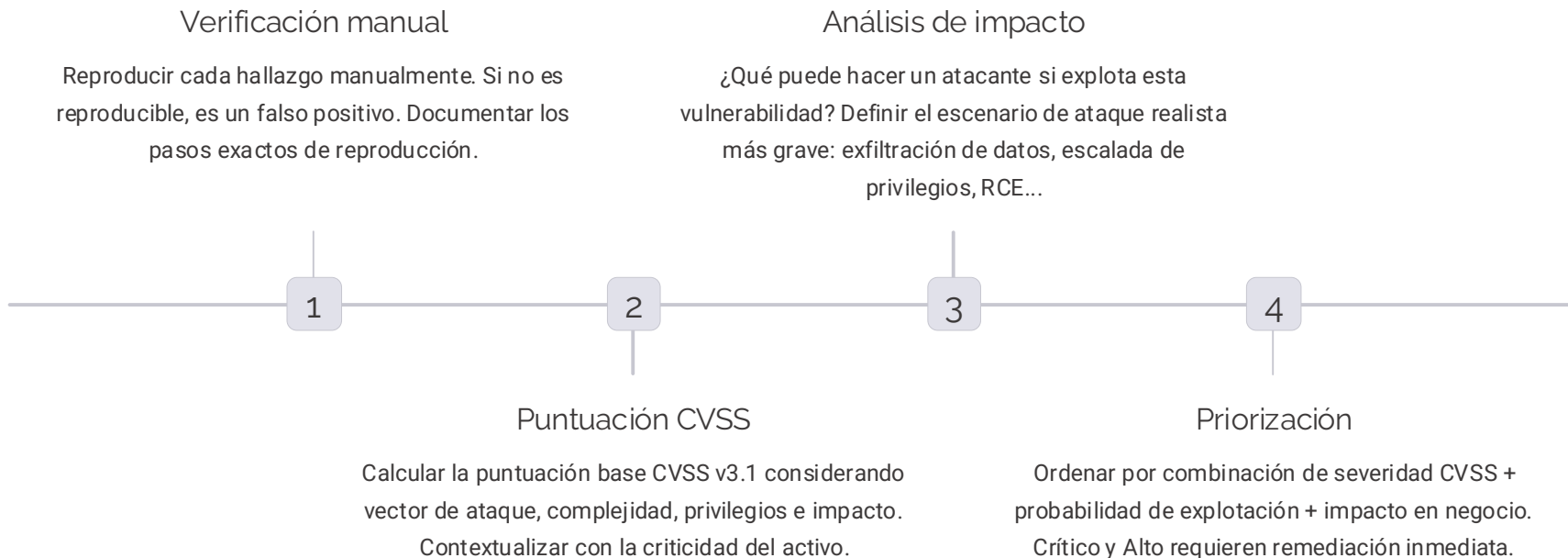
4

SSRF — Server-Side Request Forgery

Inyectar URLs internas en parámetros que la aplicación usa para hacer peticiones (`http://169.254.169.254/latest/meta-data/`). Permite acceder a metadatos de instancias cloud.

Interpretación de resultados: separar señal de ruido

Una herramienta DAST puede generar cientos de alertas. La capacidad de triaje —distinguir vulnerabilidades reales de falsos positivos y priorizar correctamente— es la competencia más valiosa del analista.



Estructura de un informe de vulnerabilidades

Un buen informe de seguridad es un documento técnico y de negocio simultáneamente. Debe ser comprensible para el desarrollador que va a remediar y para el CISO que va a priorizar inversiones.

Sección	Contenido
Resumen ejecutivo	Número de hallazgos por severidad, puntuación de riesgo global, principales conclusiones para audiencia no técnica
Metodología	Herramientas usadas, alcance, fechas, limitaciones del análisis
Hallazgo individual	Título, severidad CVSS, descripción técnica, pasos de reproducción, evidencia (capturas, payloads), impacto, recomendación
Matriz de riesgos	Tabla con todos los hallazgos, severidad, sistema afectado y estado de remediación
Anexos	Logs relevantes, configuraciones problemáticas, referencias CWE/CVE

Puntuación CVSS en la práctica

Entender cómo se calcula el CVSS v3.1 permite argumentar la priorización de remediaciones con criterio técnico objetivo. Veamos un ejemplo real para un endpoint vulnerable a SQL Injection.

Ejemplo: SQLi en endpoint de búsqueda

Vector: **AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H**

- **AV:N** — Accesible desde red (no requiere acceso físico)
- **AC:L** — Baja complejidad de ataque
- **PR:N** — Sin privilegios previos requeridos
- **UI:N** — Sin interacción del usuario
- **C:H / I:H / A:H** — Impacto alto en confidencialidad, integridad y disponibilidad

Puntuación base: 9.8 — CRÍTICA

Factores de ajuste ambiental

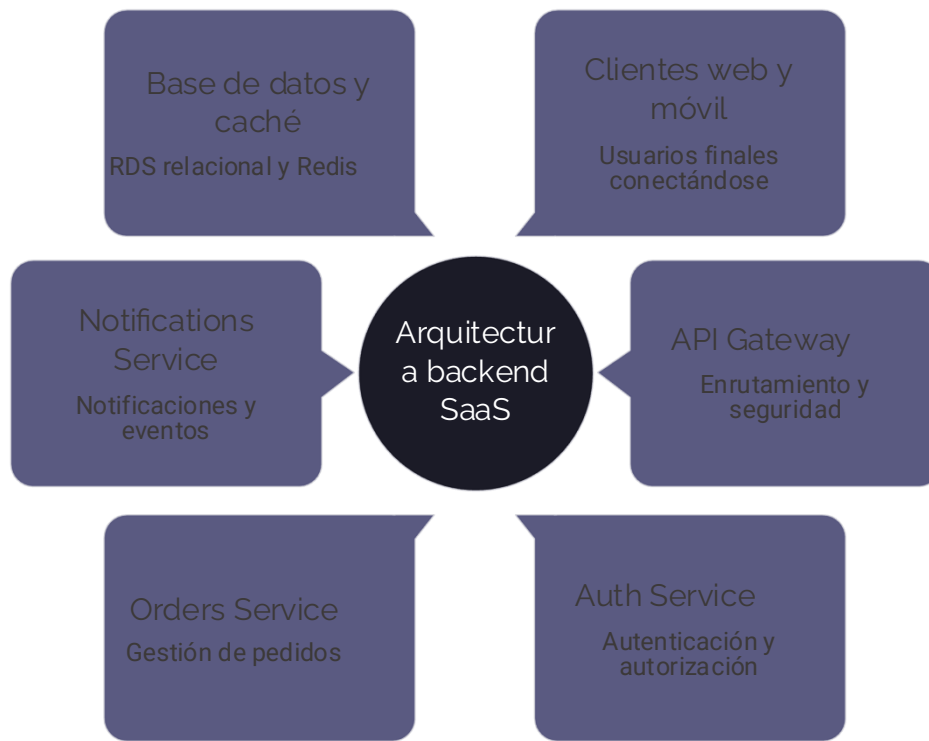
La puntuación base se ajusta con el contexto del entorno específico:

- **Criticidad del activo:** ¿contiene datos PII o financieros?
- **Exposición:** ¿es accesible desde Internet o solo desde la intranet?
- **Controles compensatorios:** ¿hay WAF, monitorización activa, segmentación de red?
- **Madurez del exploit:** ¿existe exploit público conocido?

Estos factores pueden subir o bajar la prioridad real respecto a la puntuación base.

La arquitectura bajo análisis

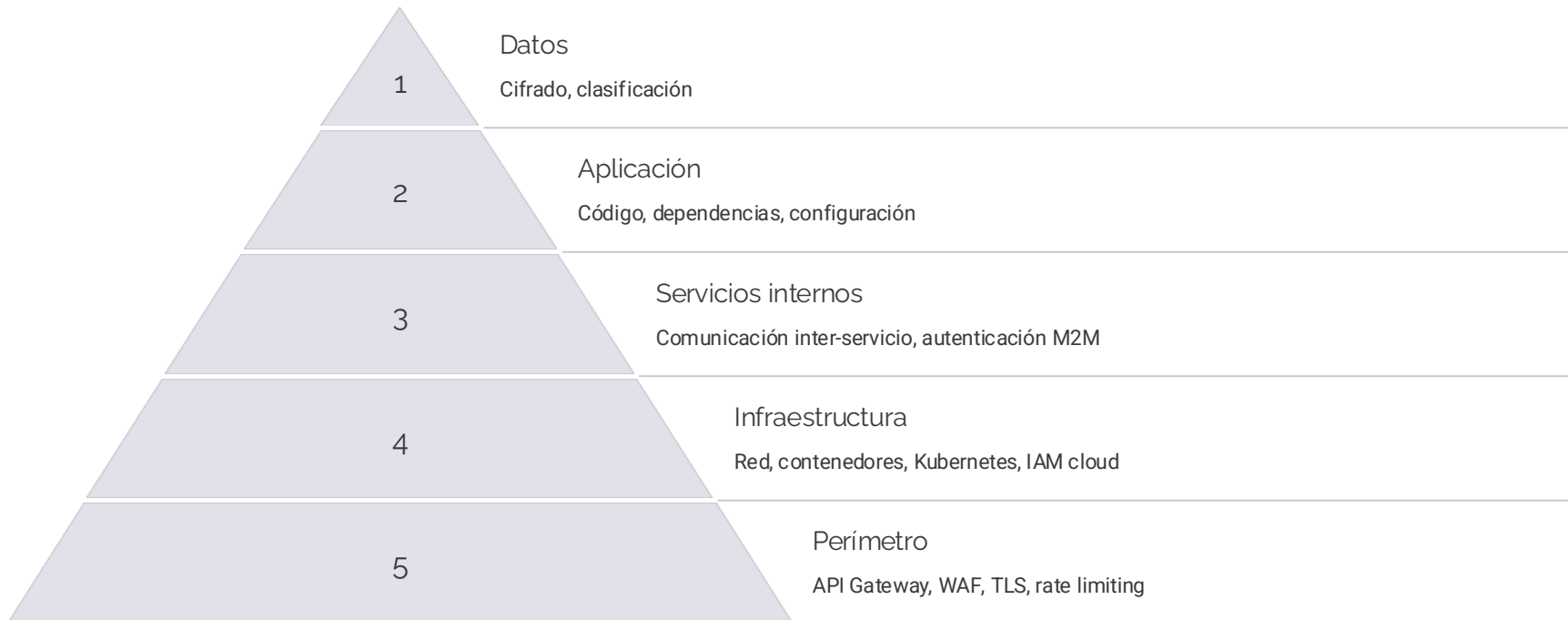
El laboratorio trabaja sobre una arquitectura backend típica de una aplicación SaaS B2B. Representa fielmente los patrones más comunes en entornos empresariales Java y .NET modernos.



Esta arquitectura presenta una superficie de ataque amplia y distribuida: múltiples endpoints públicos, servicios con diferente nivel de exposición, datos sensibles en base de datos y caché, y dependencias entre servicios que pueden ser vectores de movimiento lateral.

Metodología de revisión: las capas de análisis

La revisión de seguridad de una arquitectura backend debe ser sistemática. Organizamos el análisis en capas, desde el perímetro exterior hasta los datos, para garantizar cobertura completa sin dejar puntos ciegos.



El análisis parte del perímetro (lo que el atacante ve primero) y profundiza hacia los datos (lo más valioso). Cada capa tiene controles específicos que revisar.

Capa 1: Análisis del perímetro

El perímetro es la primera línea de defensa y la superficie más expuesta. En la arquitectura del laboratorio, el API Gateway centraliza el acceso, pero presenta múltiples puntos de configuración críticos.

TLS y certificados

Verificar versión mínima TLS 1.2 (recomendado 1.3). Revisar suite de cifrados: deshabilitar RC4, DES, 3DES, NULL. Certificado válido, no autofirmado en producción. HSTS habilitado con max-age mínimo de 1 año. Revisar con `testssl.sh` o SSL Labs.

Cabeceras de seguridad HTTP

Verificar presencia de: `Content-Security-Policy`, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy`, `Permissions-Policy`. Ausencia de `X-Powered-By` y cabeceras de servidor.

Rate limiting y throttling

¿Está configurado a nivel de API Gateway? ¿Distingue entre endpoints públicos y autenticados? ¿Tiene límites diferentes por cliente y por IP? Probar con `ab` o `hey` para verificar el comportamiento real bajo carga.

CORS

¿El `Access-Control-Allow-Origin` usa `*` en endpoints autenticados? ¿Se reflejan cabeceras `Origin` sin validación? Los endpoints que requieren cookies o tokens no deben tener CORS permisivo.

Capa 2: Infraestructura y contenedores

Los contenedores Docker y Kubernetes introducen vectores de ataque específicos que van más allá de la seguridad de la aplicación. Un contenedor mal configurado puede ser el punto de entrada para comprometer todo el cluster.

Kubernetes: riesgos críticos

- Pods corriendo como root sin `runAsNonRoot: true`
- Secrets de K8s en Base64 (no cifrado real) sin cifrado en etcd
- RBAC demasiado permisivo: roles con `*` en recursos
- Dashboard de K8s expuesto sin autenticación
- Network Policies ausentes: todos los pods se comunican entre sí
- Imágenes con vulnerabilidades conocidas (escanear con Trivy)

Docker: configuración segura

- Usar imágenes base mínimas (distroless o Alpine)
- Usuario no root en el Dockerfile (`USER appuser`)
- Sin secretos en variables de entorno ni en capas de imagen
- Filesystem de solo lectura donde sea posible
- Sin montajes de socket Docker (`/var/run/docker.sock`)
- Escanear imágenes en el pipeline con Trivy o Gype

Capa 3: Comunicación inter-servicio

En arquitecturas de microservicios, la comunicación interna entre servicios es tan importante de proteger como los endpoints externos. Un atacante que comprometa un servicio puede moverse lateralmente si la comunicación interna no está autenticada.

mTLS — Autenticación mutua

Cada servicio presenta su propio certificado al comunicarse. Garantiza que solo servicios legítimos pueden llamarse entre sí. Implementable con service mesh (Istio, Linkerd) o gestión manual de certificados.

Client Credentials OAuth2

Cada microservicio obtiene un token de acceso con alcance limitado (`scope`) para llamar a otros servicios. El token tiene tiempo de vida corto y se rota automáticamente. Flujo recomendado para M2M.

Network Policies K8s

Define explícitamente qué pods pueden comunicarse entre sí y en qué puertos. Principio de mínimo privilegio a nivel de red. Sin Network Policies, un pod comprometido puede alcanzar cualquier otro servicio del cluster.

 En el laboratorio, la arquitectura base NO tiene mTLS ni Network Policies configuradas. Esto representa un riesgo de movimiento lateral crítico que se identificará y priorizará en el análisis.

Capa 4: Análisis de la capa de aplicación

La revisión del código y la configuración de los servicios es donde se identifica la mayor densidad de vulnerabilidades en aplicaciones reales. Combinamos análisis estático con pruebas dinámicas.

1 Revisión de dependencias (SCA)

Ejecutar OWASP Dependency-Check o Snyk sobre el `pom.xml` / `.csproj`. Identificar dependencias con CVEs críticos o altos. Verificar que no hay dependencias abandonadas sin mantenimiento activo.

3 Revisión de configuración

Spring Boot actuator endpoints expuestos (`/actuator/env`, `/actuator/heapdump`). Configuración de Spring Security. Profiles de entorno. Secretos en `application.properties`.

2 Análisis estático (SAST)

Ejecutar SonarQube y Semgrep. Revisar especialmente: consultas a base de datos (SQLi), serialización/deserialización (XXE, insecure deserialization), logging de datos sensibles, uso de criptografía.

4 Pruebas dinámicas (DAST)

Escaneo con ZAP sobre el entorno de staging. Pruebas manuales con Burp Suite sobre los endpoints más críticos: autenticación, autorización, endpoints de búsqueda y carga de ficheros.

Hallazgos del laboratorio: Spring Boot Actuator expuesto

Uno de los hallazgos más frecuentes en aplicaciones Spring Boot en producción: los endpoints de Actuator expuestos sin autntfcaación revelan información sensible del entorno.

Vulnerabilidad detectada

El endpoint GET `/actuator/env` devuelve todas las propiedades del entorno, incluyendo cadenas de conexión a base de datos, claves de API y configuración de seguridad. Accesible sin autenticación desde Internet.

```
HTTP/1.1 200 OK
{
  "activeProfiles": ["prod"],
  "propertySources": [{
    "name": "applicationConfig",
    "properties": {
      "spring.datasource.url": {
        "value": "jdbc:mysql://db:3306/prod"
      },
      "jwt.secret": {
        "value": "MiSecretoJWT_Prod_2024!"
      }
    }
  ]
}
```

Remediación

Restringir los endpoints de Actuator expuestos y protegerlos con autenticación:

```
management:
  endpoints:
    web:
      exposure:
        include: health,info
  endpoint:
    health:
      show-details: never
  server:
    port: 8081
```

Exponer el puerto de gestión solo en la red interna del cluster. Nunca en la misma interfaz que la API pública. Aplicar `@Secured` o Spring Security sobre el endpoint de Actuator si se necesitan más capacidades.

Hallazgos del laboratorio: gestión insegura de tokens JWT

La implementación incorrecta de JWT es una fuente recurrente de vulnerabilidades en APIs modernas. El laboratorio revela tres problemas críticos en la gestión de tokens.

Algoritmo `none` aceptado

La librería acepta tokens con algoritmo `none`, eliminando la verificación de firma. Un atacante puede crear cualquier token con cualquier claim. **Severidad CVSS: 9.1 Crítica.** Fijar explícitamente el algoritmo permitido en la configuración de validación.

Secret débil en HS256

El secreto HMAC tiene solo 12 caracteres y aparece en el repositorio Git. Vulnerable a fuerza bruta offline con `hashcat` o `jwt-cracker`. **Severidad: Alta.** Usar secretos de al menos 256 bits, gestionados con Vault o AWS Secrets Manager.

Ausencia de validación de claims

La API no valida los claims `iss` (emisor) ni `aud` (audiencia). Un token emitido para el servicio de notificaciones es válido en el servicio de órdenes. **Severidad: Alta.** Validar siempre `iss`, `aud`, `exp` y `nbf`.

Hallazgos del laboratorio: IDOR en el servicio de órdenes

Broken Object Level Authorization (BOLA/IDOR) es consistentemente la vulnerabilidad #1 en APIs según OWASP API Security Top 10. El laboratorio muestra un caso clásico y fácilmente explotable.

```
// VULNERABLE: El ID de la orden viene directamente de la URL
// sin verificar que pertenece al usuario autenticado
@GetMapping("/api/v1/orders/{orderId}")
public ResponseEntity<Order> getOrder(
    @PathVariable Long orderId) {          // ← Sin validación de propiedad
    return orderRepository.findById(orderId)
        .map(ResponseEntity::ok)
        .orElse(ResponseEntity.notFound().build());
}

// CORRECTO: Validar que la orden pertenece al usuario autenticado
@GetMapping("/api/v1/orders/{orderId}")
public ResponseEntity<Order> getOrder(
    @PathVariable Long orderId,
    @AuthenticationPrincipal UserDetails user) {
    return orderRepository
        .findByIdAndUserId(orderId, user.getUserId()) // ← Validación de propiedad
        .map(ResponseEntity::ok)
        .orElse(ResponseEntity.notFound().build());
}
```

❌ En el entorno del laboratorio, esta vulnerabilidad permite a cualquier usuario autenticado acceder a las órdenes de cualquier otro usuario simplemente incrementando el ID en la URL. Impacto: exposición de datos de negocio de todos los clientes.

Hallazgos del laboratorio: secretos en código fuente

El escaneo del repositorio con **GitLeaks** y **TruffleHog** revela credenciales hardcodeadas en la historia de commits, un problema que persiste incluso si se "corrigen" en el commit actual.

Hallazgos en el repositorio

- Cadena de conexión a base de datos de producción en `application.properties` (commit hace 3 meses)
- API Key de Stripe hardcodeada en código de prueba, nunca eliminada
- Contraseña de cuenta de servicio AWS en script de despliegue
- Secreto JWT en fichero de configuración de test de integración

Plan de remediación

- Revocar INMEDIATAMENTE todas las credenciales expuestas, incluso las antiguas
- Reescribir la historia de Git con `git-filter-repo` para eliminar los commits problemáticos
- Implementar pre-commit hooks con `detect-secrets` para prevención
- Migrar todos los secretos a HashiCorp Vault o AWS Secrets Manager
- Configurar escaneo de secretos en el pipeline CI/CD con GitLeaks

Matriz de riesgos del laboratorio

El análisis completo de la arquitectura del laboratorio identifica los siguientes hallazgos, priorizados por severidad CVSS y urgencia de remediación:

Vulnerabilidad	CVSS	Severidad	Acción inmediata
JWT algoritmo none aceptado	9.1	Crítica	Fijar algoritmo, redeployar en < 4h
Actuator expuesto con secretos	8.6	Alta	Restringir endpoints, rotar credenciales
Secretos en repositorio Git	8.2	Alta	Revocar inmediatamente, reescribir historia
IDOR en Orders Service	7.5	Alta	Parche de código, deploy urgente
Sin Network Policies en K8s	7.1	Alta	Implementar en sprint actual
Sin mTLS inter-servicio	6.5	Media	Planificar implementación de Istio
Dependencia Log4j desactualizada	6.1	Media	Actualizar en próximo release
CORS permisivo en API pública	5.4	Media	Restringir lista de orígenes permitidos

Propuesta de mejoras: hoja de ruta priorizada

Las medidas de protección se organizan en tres horizontes temporales, desde acciones inmediatas que reducen el riesgo crítico hasta mejoras estructurales a medio plazo.

1



Inmediato (0–72 h)

- Revocar y rotar todas las credenciales expuestas
- Parchear validación JWT (fijar algoritmo, validar claims)
- Restringir Actuador a red interna
- Aplicar fix de IDOR en Orders Service

2



Corto plazo (1–4 semanas)

- Migrar secretos a HashiCorp Vault
- Implementar Network Policies en K8s
- Integrar SAST/SCA en pipeline CI/CD
- Configurar pre-commit hooks anti-secretos

3



Medio plazo (1–3 meses)

- Implementar service mesh con mTLS (Istio)
- Desplegar SIEM centralizado (ELK o Splunk)
- Establecer programa de pentesting trimestral
- Formación en seguridad para todo el equipo de desarrollo

Medidas de defensa en profundidad

La defensa en profundidad (*defense in depth*) es el principio de aplicar múltiples capas de controles de seguridad de forma que el fallo de uno no comprometa todo el sistema. Ningún control es infalible por sí solo.

Prevención

SAST, SCA, code review, formación de desarrolladores, configuración segura por defecto, gestión de secretos.

Detección

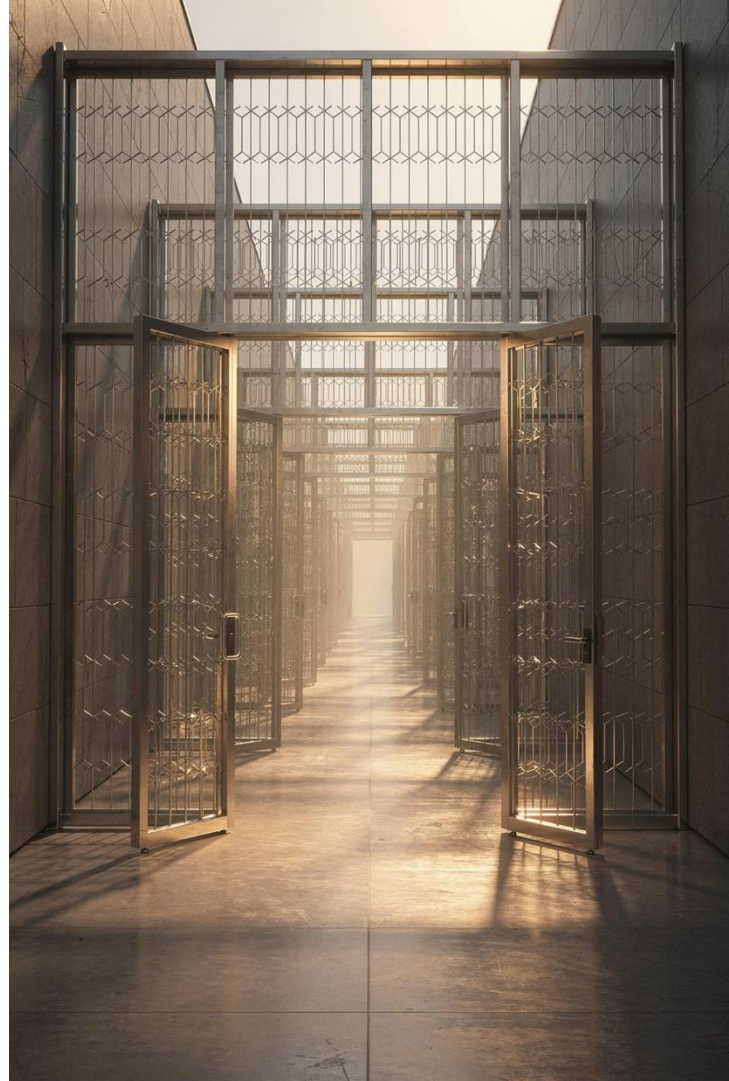
DAST en CI/CD, WAF, SIEM, anomaly detection, monitorización continua de dependencias (Dependabot).

Respuesta

Playbooks de incidentes, rotación automática de credenciales, bloqueo de IPs, invalidación de tokens, cuarentena de servicios.

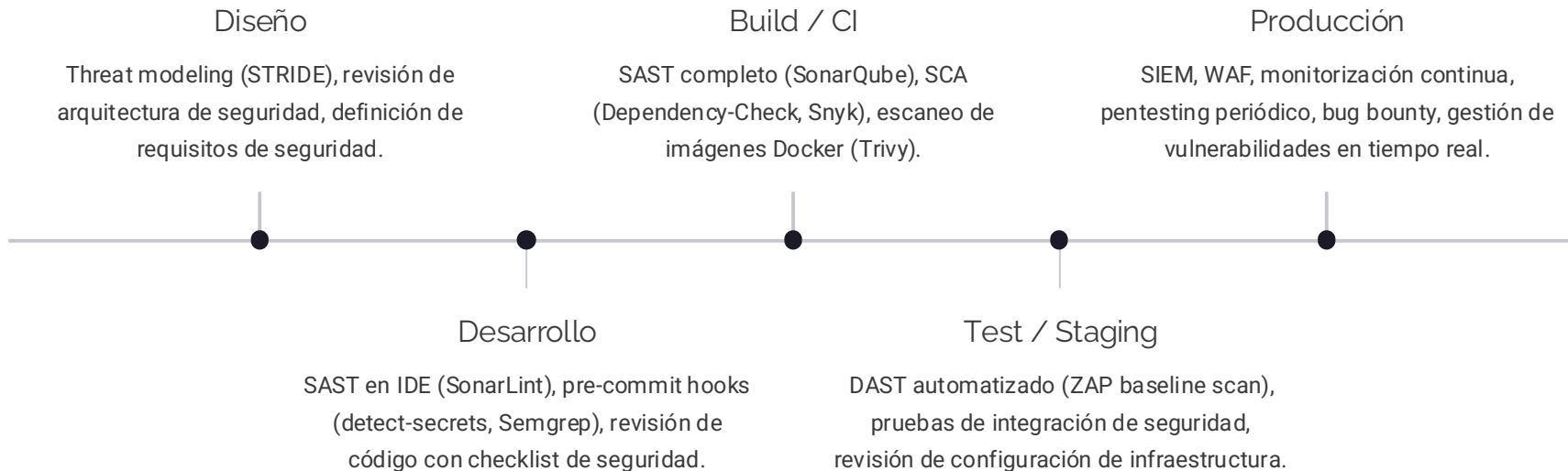
Recuperación

Backups cifrados, runbooks de restauración, comunicación de brechas (RGPD: 72h), análisis post-mortem.



Integrando la seguridad en el SDLC

El mayor retorno de inversión en seguridad se obtiene desplazando los controles hacia la izquierda del ciclo de desarrollo (*shift-left security*). Corregir una vulnerabilidad en producción cuesta hasta 100 veces más que en diseño.



Checklist de seguridad para revisiones de código

Una checklist de seguridad estandarizada en los pull requests eleva el nivel de toda la organización y convierte cada revisión de código en una oportunidad de detección temprana.

Autenticación y autorización

- ☐ ¿Todos los endpoints verifican autenticación?
- ☐ ¿Los recursos se validan contra el usuario autenticado (anti-IDOR)?
- ☐ ¿Los roles/permisos siguen el principio de mínimo privilegio?
- ☐ ¿Los tokens se validan correctamente (algoritmo, claims, expiración)?

Datos y configuración

- ☐ ¿Hay secretos, contraseñas o claves en el código o en los tests?
- ☐ ¿Las consultas a base de datos usan parámetros preparados?
- ☐ ¿Los logs no incluyen datos personales o credenciales?
- ☐ ¿Las entradas de usuario se validan y sanean antes de usarse?
- ☐ ¿Las dependencias nuevas se han revisado con SCA?

Recursos y referencias clave

Para continuar profundizando después del curso, estas son las referencias más relevantes y de mayor calidad para cada área cubierta en la sesión.



OWASP

OWASP Top 10, API Security Top 10, Testing Guide v4.2, Cheat Sheet Series. La referencia más completa y actualizada para seguridad en aplicaciones web y APIs. Descarga gratuita en owasp.org.



Certificaciones

OSCP (Offensive Security) para pentesters. **CEH** (EC-Council) para perfil generalista. **CSSLP** (ISC²) para desarrolladores con foco en SDLC seguro. **AWS/Azure Security Specialty** para arquitectos cloud.



Práctica continua

HackTheBox, TryHackMe, PortSwigger Web Security Academy – plataformas con laboratorios prácticos y progresivos. Web Security Academy de PortSwigger es especialmente recomendada para seguridad en APIs y aplicaciones web.

Resumen del módulo IV

A lo largo de las 16 horas del módulo hemos construido una visión completa de la seguridad aplicada a entornos Java y .NET: desde los fundamentos de autenticación y autorización hasta la simulación de ataques reales y la revisión de arquitecturas completas.

Sesiones 1-2

OAuth2, OIDC, JWT,
SSO, Spring Security y
ASP.NET Identity

Sesión 3

APIs seguras: rate
limiting, CORS, TLS,
headers, gestión de
secretos

Sesión 4

Vulnerabilidades
comunes: SQLi, XSS,
CSRF, XXE y sus
mitigaciones

Sesión 5

Monitorización, auditoría, simulación de ataques y laboratorio integrador



Conclusiones clave de la sesión 5



La trazabilidad es una capacidad operacional crítica

Sin logging estructurado, correlación de eventos y retención adecuada, la respuesta a incidentes es lenta, costosa e incompleta. El coste de implementarlo correctamente desde el principio es una fracción del coste de una brecha sin trazabilidad.



La seguridad debe ser continua, no puntual

Un pentest anual no es suficiente. La integración de SAST, DAST y SCA en el pipeline de CI/CD convierte cada ciclo de desarrollo en una oportunidad de detección y corrección temprana.



Los atacantes buscan el camino de menor resistencia

Las vulnerabilidades más explotadas en producción (IDOR, secretos expuestos, configuraciones inseguras) no son exploits sofisticados. Son errores evitables con buenos procesos, formación y automatización.



¡Gracias por vuestra participación!

Módulo IV completado · Seguridad Aplicada a Entornos Java & .NET

«La seguridad no es un producto, es un proceso continuo de aprendizaje, evaluación y mejora. Hoy habéis dado un paso importante en ese camino.»

¿Preguntas finales? Revisad los recursos de la sesión anterior y practicad en los laboratorios de HackTheBox y PortSwigger Web Security Academy para consolidar lo aprendido.